

Report of CUDA Bitonic Sort Kernel Optimization

Baseline: naive implementation

Global memory access (no shared memory) With outer subarr len loop and inner stride loop, complexity is $O((\log n)^2)$, for an array length of 10E8, it pads to 2^{27} , $\text{num_launches} = 27 * (27 + 1) / 2 = 378$

```
in each launch, launches
num_threads = pad_size / 2 = 2^26 threads
memory_traffic = num_threads * 2 elements * 4Byte = 512 MB read and 512 MB written per launch
```

so total memory traffic is $512\text{mb} * 378 = \sim 400\text{ GB}$ of total global memory traffic (read plus write) for a 512mb array ($2^{27} * 4\text{bytes}$) is obviously not ideal. With H100 memory bandwidth 3.35 TB/s, the memory traffic time itself is 120ms > target 80ms.

As a result, the dominate optimization is reusing data on shared memory.

Profile of baseline approach

ncu

```
==PROF== Disconnected from process 11139 [11139] a.out@127.0.0.1 Device 0, CC 9.0
bitonic_merge(int *, int, int, int) (16384, 1, 1)x(512, 1, 1), Invocations 300 Section: Command line profiler metrics
```

Metric	Name	Metric	Unit	Minimum	Maximum	Average
gpu_compute_memory_throughput.avg.pct_of_peak_sustained_elapsed		%		56.59	72.29	64.43
sm_warps_active.avg.pct_of_peak_sustained_active		%		70.30	78.18	73.84

```
fill_padding(int *, int, int, int) (52947, 1, 1)x(128, 1, 1), Invocations 1
Section: Command line profiler metrics
```

Metric	Name	Metric	Unit	Minimum
Maximum	Average			
gpu_compute_memory_throughput.avg.pct_of_peak_sustained_elapsed		%		18.62
sm_warps_active.avg.pct_of_peak_sustained_active		%		9.65

grade.py result

Achieved Occupancy: 41.83 Memory Throughput: 41.52

FUNCTIONAL SUCCESS Array size : 100000000 CPU Sort Time (ms) : 14881.471680 GPU Sort Time (ms) : 296.540955 GPU Sort Speed : 337.221558 million elements per second PERF PASSING GPU Sort is 50x faster than CPU !!! H2D Transfer Time (ms): 42.335297 Kernel Time (ms) : 123.914749 D2H Transfer Time (ms): 130.290909

Kernel Time: 123.787903ms, Score: 6.463 Memory Transfer Time: 172.434749ms, Score: 0.696 Million elements per second: 337.584 Total Score: 13.16 pts

Major optimization: tiling through shared memory

By tiling, tile size T of threads load data to shared memory once and reuse

Host Launch Algorithm

Pad array size, allocate and move data to device global memory

Pad array size 100M to nearest power of two, $\text{pad_size} = 2^{27}$.

Decide launch grid and tile size

H100 shared memory is 48KB (software default), $48\text{KB} / 4\text{Byte} = 12\text{K}$, tile size is the nearest power of two $\leq 12\text{K}$ is 8192. Giving per thread block limitation 1024 threads, set $\text{threads_per_block} = 1024$, and $\text{blocks_per_grid} = \text{pad_size} / 2^{27} / 1024$.

Launch kernelA for $k \leq 8192$

In k ranges $[2, 4, \dots N]$, when $k \leq 8192$, the shared memory can hold all k loops and stride loops with fixed tile size 8192.

Launch kernelB for $k > 8192$

Clear device memory

Device Kernel A Algorithm

Phasing loop loading

Each thread need to load $8192 / 1024 = 8$ elements to shared memory, so the phasing loop need 8 loops for 1024 threads to load entire tile data. Each thread load stride is 1024 for memory coalescing.

Sync threads after loading tile data.

Loop k and stride

Loop k in $[2^1, \dots 2^{13}]$, inner loop stride in $[k/2, k/4 \dots 1]$, sequentially compare and swap $\text{arr}[i]$ with $\text{arr}[i + \text{stride}]$.

Each thread does $4096 / 1024 = 4$ pairs, with stride 1024 (eg: thread $[t]$ works on $\text{arr}[t]$, $\text{arr}[t + 1024]$, $\text{arr}[t + 2048]$, $\text{arr}[t + 3072]$, and corresponding $\text{arr}[i + \text{stride}]$).

Sync threads after each stride done.

Write back to global memory

Device Kernel B Algorithm

Each kernel takes in current k for direction, fix tile size = 8192, loop over all strides in $[4096, 1]$ inside kernel. Launch grid: $\langle \langle \rangle \rangle$

Phasing loop loading

Loop phase in $[0, \text{TILE} / 1024 - 1]$. Sync threads.

Loop stride and compare

For stride in $[4096, 1]$, each thread compare $\text{num_pairs} = 4096 / 1024$, phase loop: derive i , compare and swap.

Sync threads after each stride done.

Write back to global memory

Memory transfer optimization

Profile

```
ncu
==PROF==  Disconnected from process 17772 [17772] a.out@127.0.0.1 Device 0, CC 9.0
bitonic_merge(int *, int, int, int) (16384, 1, 1)x(512, 1, 1), Invocations 66 Section: Command line
profiler metrics -----
Metric      Name      Metric      Unit      Minimum      Maximum      Average
-----
gpu_compute_memory_throughput.avg.pct_of_peak_sustained_elapsed % 56.18 66.77 62.69
sm_warps_active.avg.pct_of_peak_sustained_active % 74.99 78.49 75.62
-----

bitonic_merge_large_k(int *, int) (2048, 1, 1)x(1024, 1, 1), Invocations 11
Section: Command line profiler metrics
-----
Metric Name      Metric Unit Minimum
Maximum Average
-----
gpu_compute_memory_throughput.avg.pct_of_peak_sustained_elapsed %
30.94 32.90 31.18
sm_warps_active.avg.pct_of_peak_sustained_active %
96.29 96.31 96.30
-----

bitonic_merge_small_k(int *) (2048, 1, 1)x(1024, 1, 1), Invocations 1
Section: Command line profiler metrics
-----
Metric Name      Metric Unit Minimum
Maximum Average
-----
gpu_compute_memory_throughput.avg.pct_of_peak_sustained_elapsed %
32.77 32.77 32.77
sm_warps_active.avg.pct_of_peak_sustained_active %
49.93 49.93 49.93
-----

fill_padding(int *, int, int, int) (52947, 1, 1)x(128, 1, 1), Invocations 1
Section: Command line profiler metrics
-----
Metric Name      Metric Unit Minimum
Maximum Average
-----
gpu_compute_memory_throughput.avg.pct_of_peak_sustained_elapsed %
18.63 18.63 18.63
sm_warps_active.avg.pct_of_peak_sustained_active %
9.59 9.59 9.59
-----
```

grade.py

Achieved Occupancy: 57.99 Memory Throughput: 36.27

Kernel Time: 73.472351ms, Score: 10 Memory Transfer Time: 170.83196999999998ms, Score: 0.702
Million elements per second: 409.326 Total Score: 16.7 pts

After kernel shared memory optimization, I got a result with H2D, kernel time looking ok, but D2H takes too much time.

Initially I thought the reason is naive malloc allocating physical pages lazily on every page size device copy to CPU, and there's one OS page fault overhead to allocate physical page. So I tried to replace naive malloc with cudaHostAlloc, using pin memory. The mechanism is it will allocate a locked pin memory on host memory, and device can directly copy to that area which is the time win.

However, using cudaHostAlloc barely changes D2H transfer time. So reasoning through, I realize there's no free lunch, cudaHostAlloc takes similar time allocating memory as naive malloc do, the actual mechanism is that cudaHostAlloc takes as long time as malloc to allocate but once done, device can copy faster, while for pageable memory and malloc, each D2H copy takes longer time, since device need to copy data to a small pinned memory buffer on host, and host need extra memcopy to move data to actual allocated array location, which cudaHostAlloc reduces. [2] alt text

Furthermore, cudaHostAlloc reduces time by overlapping with kernel running, while CPU is idle. [2] There's syncthreads before and after bitonic_sort() in main.cu, so my allocating with cudaHostAlloc in host_to_dev() makes it unable to simultaneously work with GPU, by moving allocation into bitonic_sort() reduced D2H greatly.

cudaHostAlloc reference documentation: [1] https://docs.nvidia.com/cuda/cuda-runtime-api/group__CUDART__MEMORY.html#group__CUDART__MEMORY_1gb65da58f444e7230d3322b6126bb4902

[2] <https://developer.nvidia.com/blog/how-optimize-data-transfers-cuda-cc/>

Other optimizations

1. Data type optimization

arrCpu[i] = rand() % 1000; in main.cu indicates that value inside array ranges in [0, 999], so replacing DTYPE int -> uint16_t / int16_t / short which is reducing each data size from 32bit -> 16bit, halving the data transferred.

Also, with 2Byte per element, 48KB shared memory can also allow larger tile size, 48KB / 2Byte = 24K, tile size is the nearest power of two <= 24K is 16384. threads_per_block is still 1024 threads, and blocks_per_grid = pad_size 2^27 / 1024.

grade.py profile

Achieved Occupancy: 69.44 Memory Throughput: 22.88

FUNCTIONAL SUCCESS Array size : 100000000 CPU Sort Time (ms) : 15084.942383 GPU Sort Time (ms) : 86.006721 GPU Sort Speed : 1162.699829 million elements per second PERF PASSING GPU Sort is 175x faster than CPU !!! H2D Transfer Time (ms): 21.131489 Kernel Time (ms) : 61.237247 D2H Transfer Time (ms): 3.637984

Kernel Time: 61.164257ms, Score: 10 Memory Transfer Time: 24.769472999999998ms, Score: 4
Million elements per second: 1163.687 Total Score: 21 pts

2. Move data on register rather than read from global

```
"" // before - double load each from global memory
```

```
if ((arr[i] < arr[i + stride]) != dir) { // swap DTYPE tmp = arr[i]; arr[i] = arr[i + stride]; arr[i + stride] = tmp; }
```

```
// after - once load from global memory DTYPE a = arr[i]; DTYPE b = arr[i + stride]; if ((a < b) != dir)
{ arr[i] = b; arr[i + stride] = a; }
'''
```

ncu profile

A. Overall kernel cost

Kernel Name	Number of Launches	Total Duration (milliseconds)	Percentage of Total Kernel Time	Achieved Occupancy (percent)	Theoretical Occupancy (percent)	Memory Throughput (percent of Speed of Light)	Divergent Branches
bitonic_merge_small_k	1	2.155	26.1%	49.93%	50.0%	32.77%	0
bitonic_merge	66	2.709	32.8%	75.57%	100.0%	62.64%	0
bitonic_merge_large_k	11	3.367	40.7%	96.30%	100.0%	31.17%	0
fill_padding	1	0.034	0.4%	9.58%	100.0%	18.61%	0
Total	79	8.266	100.0%	—	—	—	0

(The max occupancy of bitonic_merge_small_k caps at 50% because with 33.8 KB of shared memory per block, only one block fits per SM, its thread_per_block = 1024 = 32 warps, out of the H100's 64-warp capacity is 50%.)

B. Memory

Kernel Name	Global Memory Load Sectors (M)	Global Memory Store Sectors (M)	Global Memory Read (GiB)	Global Memory Written (GiB)	L2 Hit Rate	Shared Memory Load Wavefronts (M)	Shared Memory Store Wavefronts (M)	Shared Memory Bank Conflicts (M)	Bank Conflicts per Wavefront
bitonic_merge_small_k	2.1	2.1	0.07	0.04	18.0%	77.1	57.9	50.2	0.37
bitonic_merge	138.4	66.9	4.33	1.23	15.4%	0.0	0.0	0.0	—
bitonic_merge_large_k	23.1	23.1	0.72	0.48	18.7%	110.2	66.1	45.9	0.26
fill_padding	0.0	0.8	~0	~0	—	0.0	0.0	0.0	—
Total	163.6	92.9	5.11	1.75	16.4%	187.3	124.1	96.0	—

A. Overall kernel cost (after data type and tile size optimization: DTYPE = short, TILE = 16384, 10M elements)

Kernel Name	Number of Launches	Total Duration (milliseconds)	Percentage of Total Kernel Time	Achieved Occupancy (percent)	Theoretical Occupancy (percent)	Memory Throughput (percent of Speed of Light)	Divergent Branches
bitonic_merge_small_k	1	2.137	29.9%	93.95%	100.0%	25.08%	0
bitonic_merge	55	2.004	28.0%	80.16%	100.0%	30.14%	0
bitonic_merge_large_k	10	2.981	41.7%	93.79%	100.0%	26.96%	0
fill_padding	1	0.034	0.5%	9.65%	100.0%	9.58%	0
Total	67	7.156	100.0%	—	—	—	0

B. Memory (after data type and tile size optimization: DTYPE = short, TILE = 16384, 10M elements)

Kernel Name	Global Memory Load Sectors (M)	Global Memory Store Sectors (M)	Global Memory Read (GiB)	Global Memory Written (GiB)	L1 Hit Rate	L2 Hit Rate	Shared Memory Load Wavefronts (M)	Shared Memory Store Wavefronts (M)	Shared Memory Bank Conflicts (M)	Bank Conflicts per Wavefront
bitonic_merge_small_k	1.0	1.1	0.03	0.01	46.8%	51.2%	55.6	41.3	0.034	0.0003
bitonic_merge	57.7	27.7	1.80	0.17	27.1%	33.9%	0.0	0.0	0.000	—
bitonic_merge_large_k	10.5	10.5	0.33	0.13	46.6%	51.2%	78.8	47.5	0.297	0.0024

Kernel Name	Global Memory Load Sectors (M)	Global Memory Store Sectors (M)	Global Memory Read (GiB)	Global Memory Written (GiB)	L1 Hit Rate	L2 Hit Rate	Shared Memory Load Wavefronts (M)	Shared Memory Store Wavefronts (M)	Shared Memory Bank Conflicts (M)	Bank Conflicts per Wavefront
fill_padding	0.0	0.4	0.00	0.00	48.3%	99.8%	0.0	0.0	0.000	—
Total	69.2	39.7	2.16	0.31	30.3%	36.8%	134.4	88.8	0.331	0.0015

From A, from low memory throughput and the low compute occupancy of the algorithm, we can find that

Profiling command and metric mapping

Table A

- Number of Launches - row count per kernel in the export (reported by ncu as Invocations)
- Total Duration - `gpu__time_duration.sum`, summed over all launches of the kernel
- Percentage of Total Kernel Time - derived from `gpu__time_duration.sum`
- Achieved Occupancy - `sm__warps_active.avg.pct_of_peak_sustained_active`
- Theoretical Occupancy - `sm__maximum_warps_per_active_cycle_pct`
- Memory Throughput - `gpu__compute_memory_throughput.avg.pct_of_peak_sustained_elapsed`
- Divergent Branches - `smsp__sass_branch_targets_threads_divergent.sum`

Table B

- Global Memory Load Sectors - `lltex__t_sectors_pipe_lsu_mem_global_op_ld.sum`
- Global Memory Store Sectors - `lltex__t_sectors_pipe_lsu_mem_global_op_st.sum`
- Global Memory Read - `dram__bytes_read.sum` (traffic that reached DRAM, i.e. after L1 and L2 absorb hits, so it is lower than the load sector count)
- Global Memory Written - `dram__bytes_write.sum`
- L1 Hit Rate - `lltex__t_sector_hit_rate.pct`
- L2 Hit Rate - `lts__t_sector_hit_rate.pct`
- Shared Memory Load Wavefronts - `lltex__data_pipe_lsu_wavefronts_mem_shared_op_ld.sum`
- Shared Memory Store Wavefronts - `lltex__data_pipe_lsu_wavefronts_mem_shared_op_st.sum`
- Shared Memory Bank Conflicts - `lltex__data_bank_conflicts_pipe_lsu_mem_shared.sum`, which the export confirms equals `lltex__data_bank_conflicts_pipe_lsu_mem_shared_op_ld.sum` plus `lltex__data_bank_conflicts_pipe_lsu_mem_shared_op_st.sum`
- Bank Conflicts per Wavefront - derived as bank conflicts divided by the sum of shared load and store wavefronts

Local memory accesses are not given a column because `lltex__t_sectors_pipe_lsu_mem_local_op_ld.sum` and `lltex__t_sectors_pipe_lsu_mem_local_op_st.sum` are both 0 for every kernel, confirming no register spilling.

Hit rates are per-launch averages within each kernel; the Total row weights them by global load sectors, since percentages cannot be summed. Metrics named above that are not in the explicit --

metrics list (gpu__time_duration.sum, gpu__compute_memory_throughput..., dram__bytes_*, the shared wavefront counters) come from --set full.

Bugs

1. Out of bound when pad_size < TILE=8192 with optimized kernels

Initially in optimized